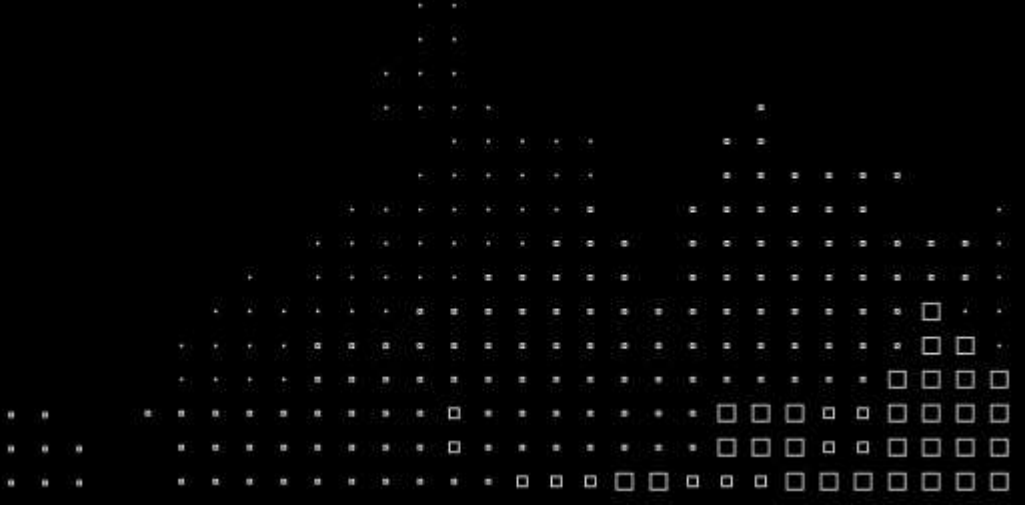
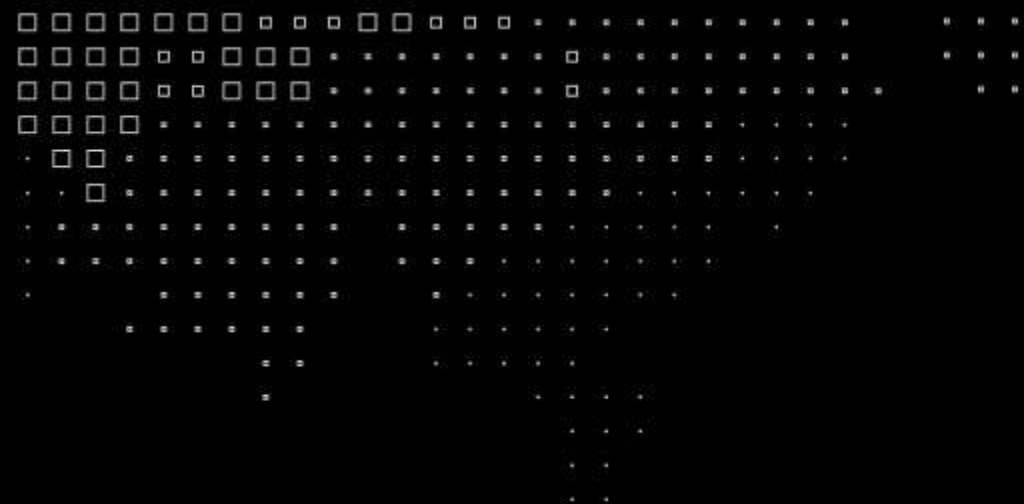


graduação⁺

ENGENHARIAS

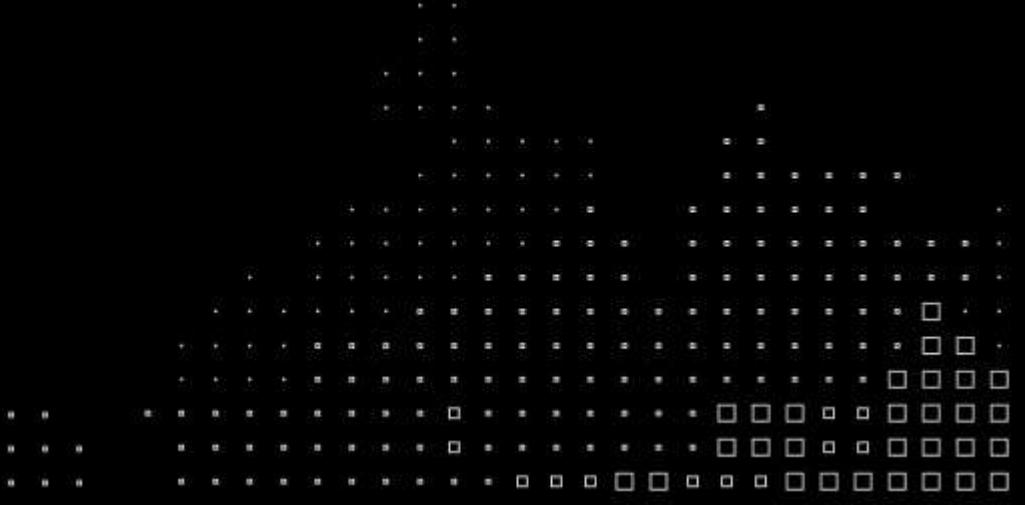




graduação⁺

SISTEMAS CIBER FÍSICOS

Prof.: Deimonril Gomes da Silva (Hill)
Email: profdeimonril.silva@fiap.com.br



O Preço da Conectividade em Sistemas Ciber-Físicos

Você foi contratado para projetar o sistema de ignição sem fio de um foguete de sondagem. O cliente quer disparar o motor via Wi-Fi pelo celular. Como você estrutura o hardware?

A: Um ESP32 cuidando do servidor Web e do pino de disparo simultaneamente (via RTOS).

B: Um roteador Wi-Fi comercial cabeado a um microcontrolador.

C: Um ESP32 apenas para a rede, que envia um pulso elétrico para um Arduino Uno dedicado exclusivamente à contagem regressiva.

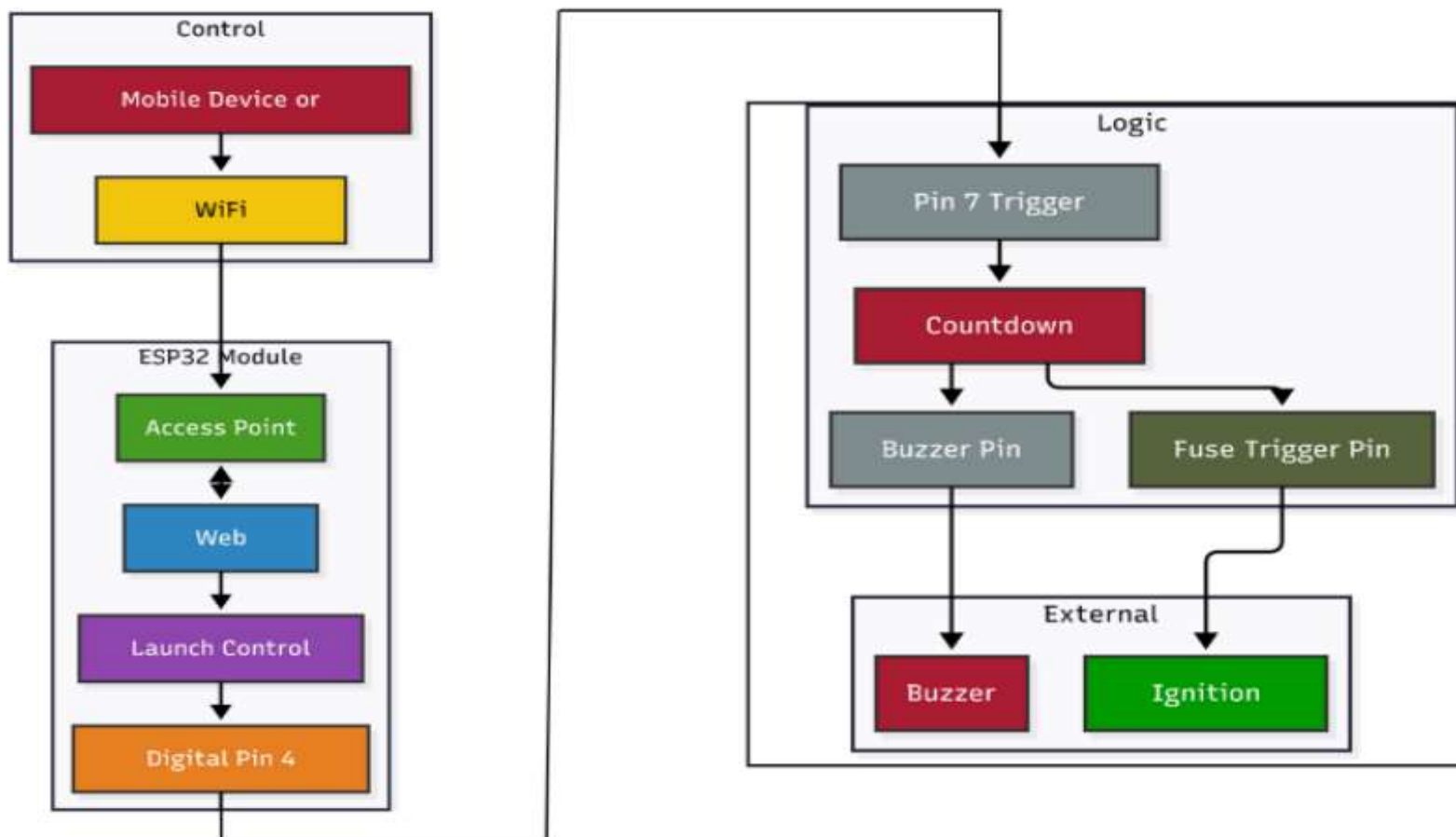
Você foi contratado para projetar o sistema de ignição sem fio de um foguete de sondagem. O cliente quer disparar o motor via Wi-Fi pelo celular. Como você estrutura o hardware?

A: Um ESP32 cuidando do servidor Web e do pino de disparo simultaneamente (via RTOS).

B: Um roteador Wi-Fi comercial cabeado a um microcontrolador.

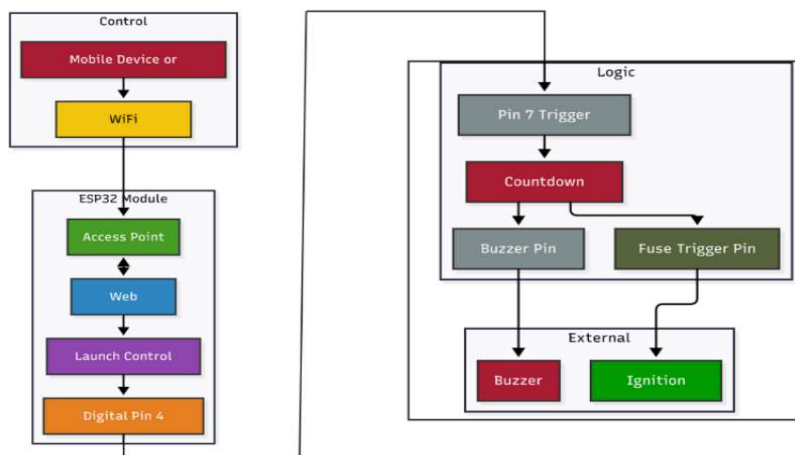
***C:* Um ESP32 apenas para a rede, que envia um pulso elétrico para um Arduino Uno dedicado exclusivamente à contagem regressiva.**

Discussão (Apresentando a Solução):



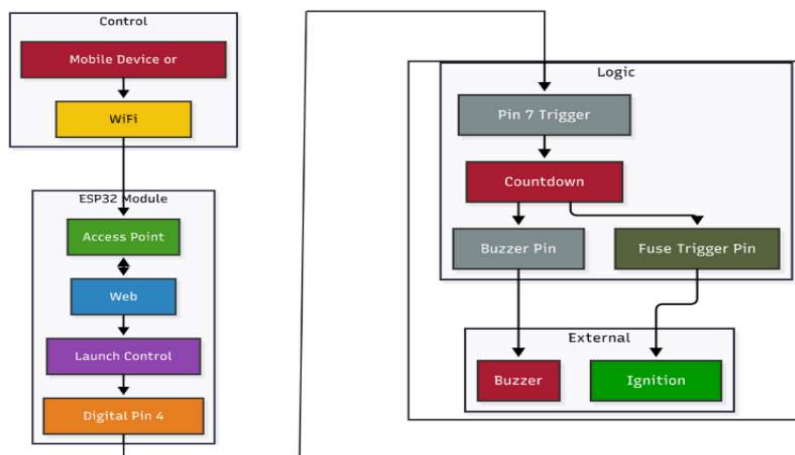
(Panda et al., 2026)

Discussão (Apresentando a Solução):



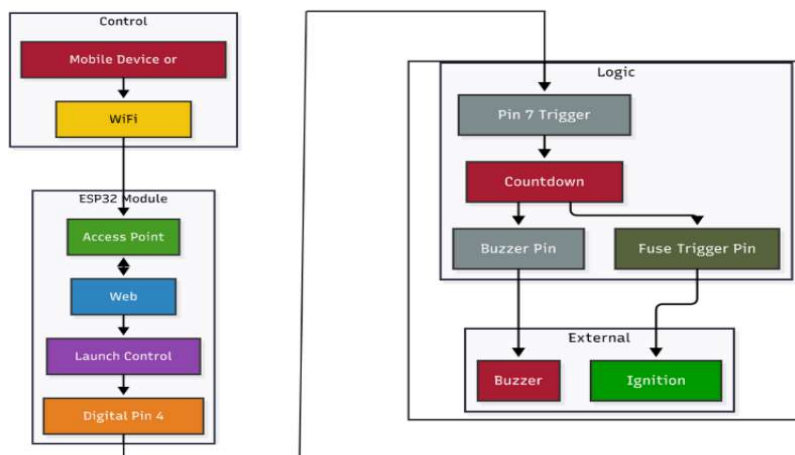
Problema Comum: Falhas em sistemas de ignição remota com microcontrolador único decorrentes de concorrência entre rede e controle.

Discussão (Apresentando a Solução):



Solução Proposta: Arquitetura dual com divisão clara de responsabilidades (ESP32 + Arduino UNO)

Discussão (Apresentando a Solução):



ESP32: Hub de comunicação Wi-Fi, servidor web e autenticação de segurança.

Arduino UNO: Execução determinística da contagem regressiva e acionamento direto de hardware.

13

14

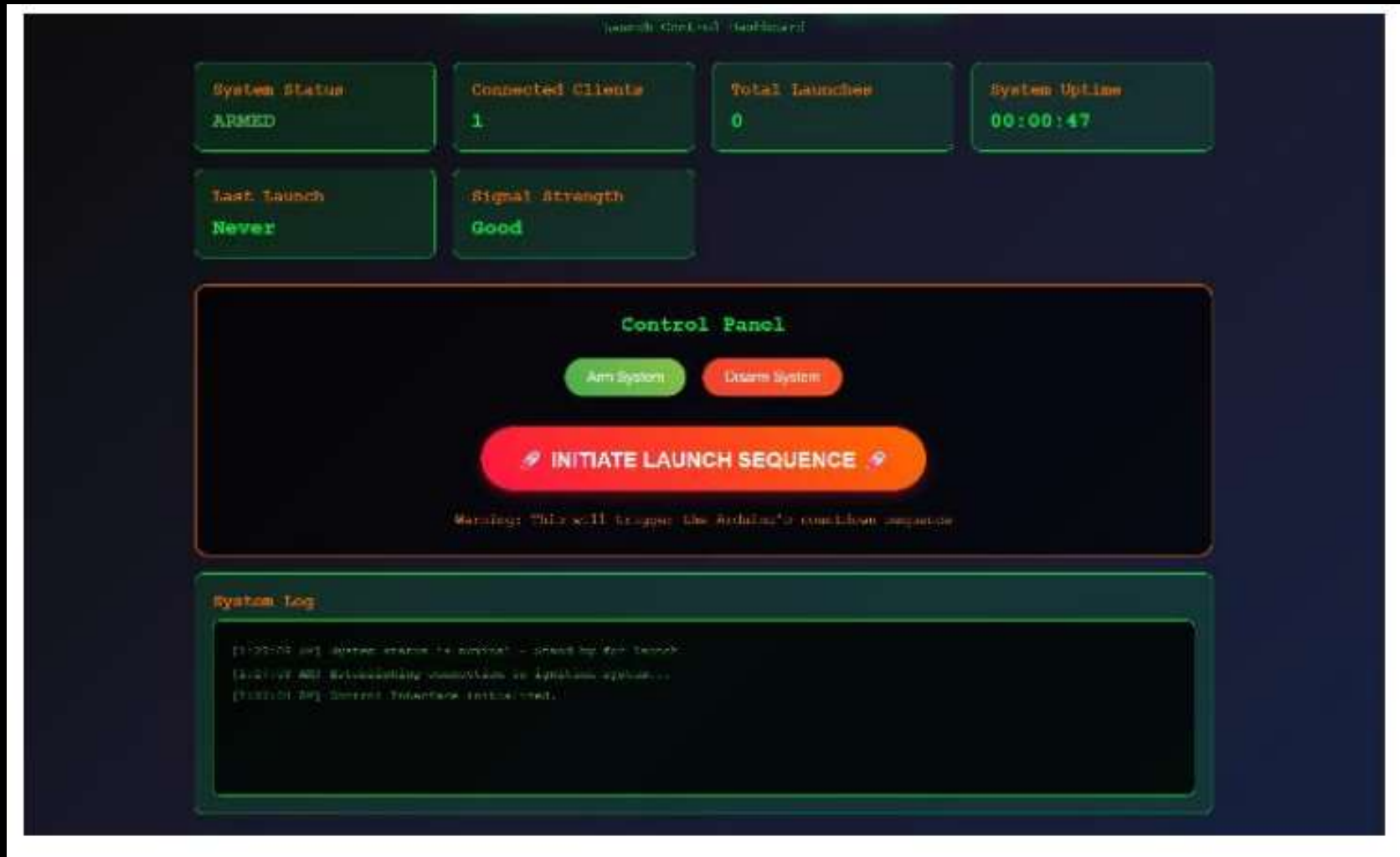
```
#include <WiFi.h>
```

15

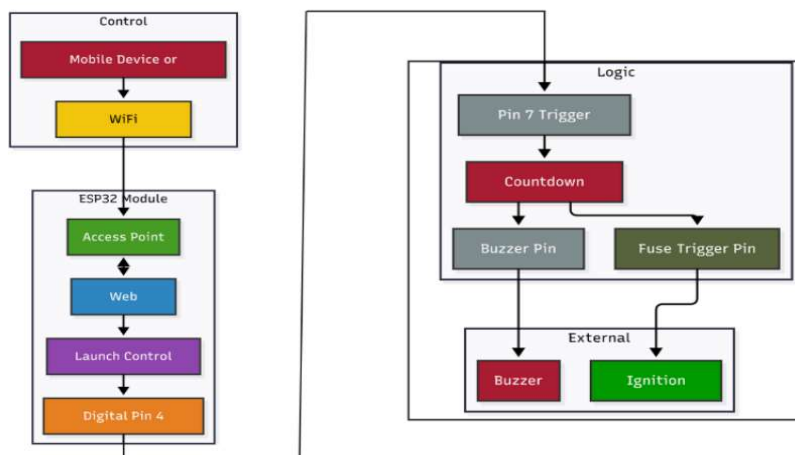
```
#include <WebServer.h>
```

16

Dashboard apresentado pelo autor



Discussão (Apresentando a Solução):

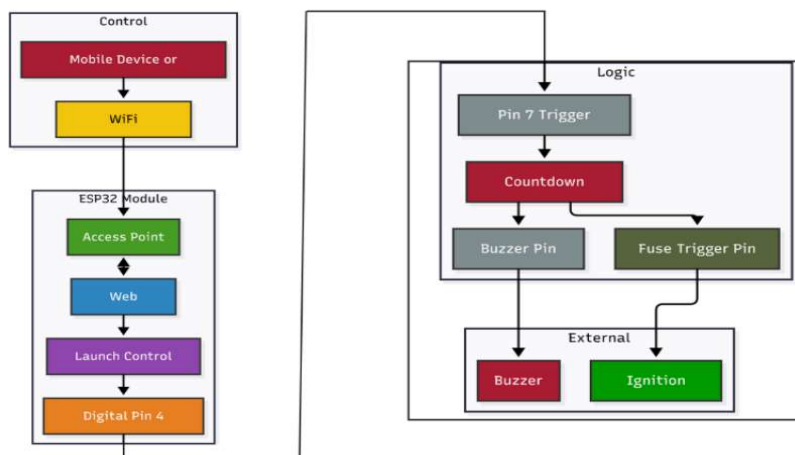


Access Point Nativo: Criação de rede Wi-Fi própria, sem dependência de infraestrutura externa.

```
29  const char* AP_SSID = "FOGUETE_CONTROL_AP";  
30  const char* AP_PASS = "Ignition2026"; // Mínimo de 8 caracteres (WPA2-PSK)
```

```
132  Serial.println("\nIniciando Access Point Wi-Fi...");  
133  WiFi.softAP(AP_SSID, AP_PASS);  
134  
135  IPAddress apIP = WiFi.softAPIP();  
136  Serial.print("Access Point Criado com Sucesso!");  
137  Serial.print(" Endereço IP do Painel: ");  
138  Serial.println(apIP);
```

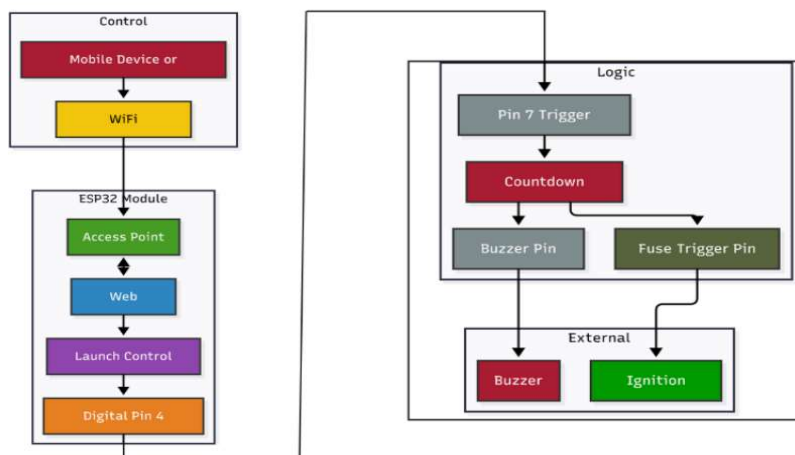
Discussão (Apresentando a Solução):



Painel Web (Dashboard): Acesso via navegador (móvel ou desktop) para monitoramento em tempo real.

```
48 void handleRoot() {
49     String html = "<!DOCTYPE html><html><head><meta name='viewport' content='width=device-width, initial-scale=1.0'>";
50     html += "<title>Painel de Controle de Ignição</title>";
51     html += "<style>body{font-family:Arial;background:#121212;color:#fff;text-align:center;padding:20px;}";
52     html += ".btn{padding:15px 30px;margin:10px;font-size:18px;border:none;border-radius:5px;cursor:pointer;}";
53     html += ".arm{background-color:#f39c12;color:#white;} .disarm{background-color:#27ae60;} .launch{background-color:#c0392b;}";
54     html += ".status-box{border:1px solid #333;padding:15px;margin:20px auto;max-width:400px;border-radius:8px;}</style></head><body>";
55
56     html += "<h2>Painel de Ignição Remota (ESP32)</h2>";
57     html += "<div class='status-box'>";
58     html += "<p>Estado do Sistema: <b>" + String(systemArmed ? "<span style='color:#e67e22;'>ARMADO</span>" : "<span style='color:#2ecc71;'>DESARMADO</span>");
59     html += "<p>Lançamentos Efetuados: " + String(launchCount) + "</p>";
60     html += "<p>Tempo de Atividade (Uptime): " + String(millis() / 1000) + "s</p>";
61     html += "</div>";
62
63     // Botões de Ação
64     if (!systemArmed) {
65         html += "<a href='/arm'><button class='btn arm'>ARMAR SISTEMA</button></a>";
66     } else {
67         html += "<a href='/disarm'><button class='btn disarm'>DESARMAR SISTEMA</button></a>";
68         html += "<br><br><a href='/launch'><button class='btn launch'>INICIAR SEQUÊNCIA DE DISPARO</button></a>";
69     }
70
71     html += "</body></html>";
72     server.send(200, "text/html", html);
73 }
```

Discussão (Apresentando a Solução):



Protocolos e APIs: Requisições REST-like para armar, desarmar e validar o disparo.

Uso Eficiente de Recursos: Utiliza apenas 10% de seus pinos GPIO (GPIO 4, 16 e 17).

>No HTML do server (ESP32) temos 3 links que se apresentarão como botões

>Cada um desses links vai gerar uma requisição, direcionando para /arm, /disarm, /launch

```
64  if (!systemArmed) {
65      html += "<a href='/arm'><button class='btn arm'>ARMAR SISTEMA</button></a>";
66  } else {
67      html += "<a href='/disarm'><button class='btn disarm'>DESARMAR SISTEMA</button></a>";
68      html += "<br><br><a href='/launch'><button class='btn launch'>INICIAR SEQUÊNCIA DE DISPARO</button></a>";
69  }
70
```

>Isso permite que nossa tabela de rotas entre em ação, com os handles sendo responsáveis por "rotear"

>Já na void loop() temos um polling handleClient() que, a todo tempo, verifica as requisições na porta 80, vindas do Client pra redirecionar para a rotina correta

```
141  server.on("/", handleRoot);
142  server.on("/arm", handleArm);
143  server.on("/disarm", handleDisarm);
144  server.on("/launch", handleLaunch);
```

```
154  void loop() {
155      // [SLIDE 2]: Escuta e atende :
156      server.handleClient();
157
```

>Dentro de handle.launch, a primeira etapa é utilizar essa “máquina de estados” pra verificar se, de fato o sistema está armado antes de permitir o lançamento.

```
94     if (!systemArmed) {  
95         server.send(403, "text/plain", "ERRO: O sistema precisa estar ARMADO para disparar!");  
96         return;  
97     }
```

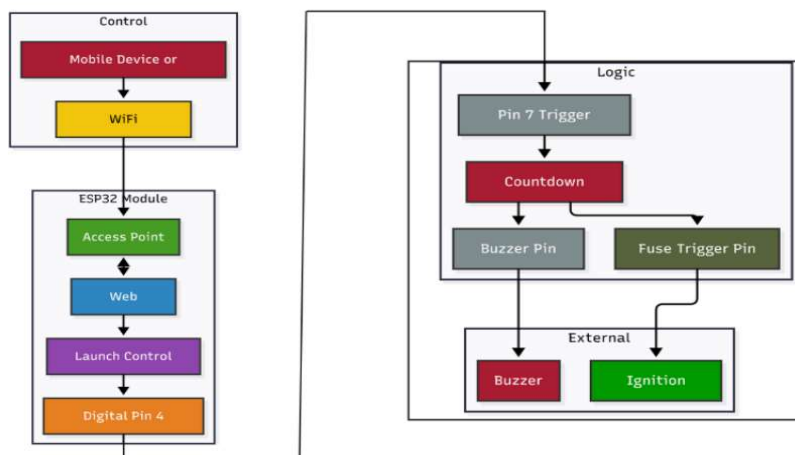
- > Mantém o pino do Arduino acionado por um tempo estendido pra garantir a leitura.
- > Atualiza as informações que serão enviadas para o Dashboard
- > Envia o texto confirmando

```
101     digitalWrite(PIN_TRIGGER_ARDUINO, LOW);  
102     delay(200); // Pul  
103     digitalWrite(PIN_TRIGGER_ARDUINO, HIGH); // R  
104
```

```
106     launchCount++;  
107     systemArmed = false;  
108     digitalWrite(PIN_LED_STATUS, LOW);  
109
```

```
110     // Retorna resposta de confirmação de envio de comando  
111     server.send(200, "text/plain", "Sinal de disparo enviado com sucesso ao Arduino UNO!");  
112 }
```

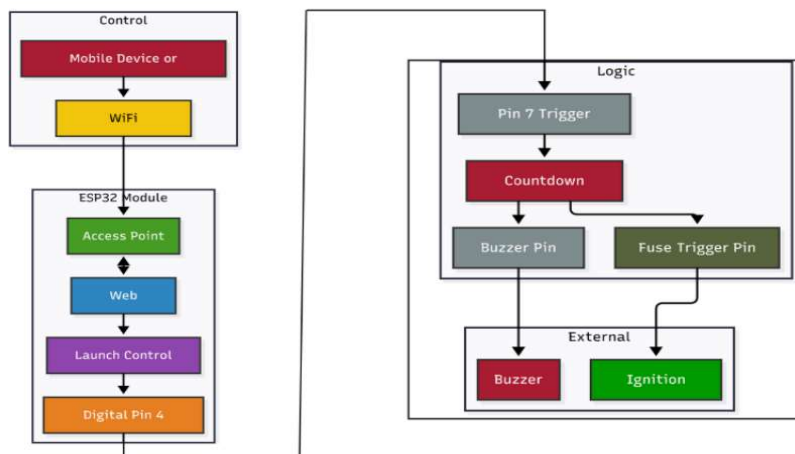
Discussão (Apresentando a Solução):



Controle em Tempo Real: Execução do protocolo de contagem regressiva em três fases.

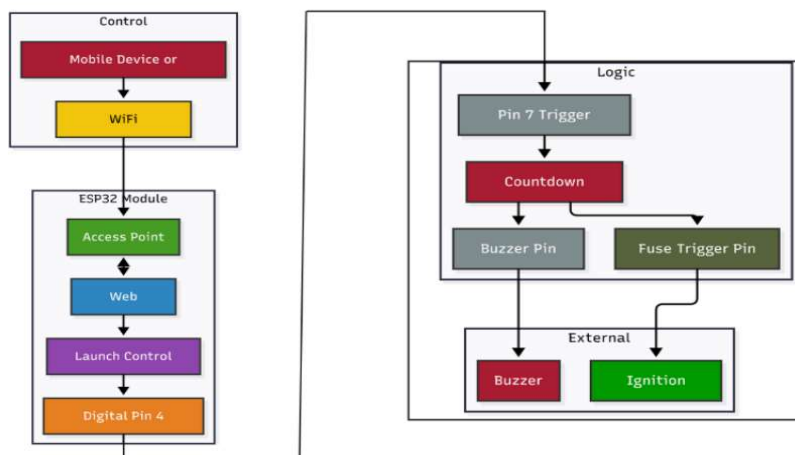
Feedback Sonoro: Buzzer no Pino 13 com variação de frequência (alertas progressivos).

Discussão (Apresentando a Solução):



Atuador Elétrico: Pino 9 (saída PWM com 100% *duty cycle* por 3 segundos para o pavio/estopim).
Pino de Disparo Estável: Pino 7 configurado em **INPUT_PULLUP** para evitar gatilhos falsos por ruído elétrico.

Discussão (Apresentando a Solução):



Mapeamento do Link:

GPIO 4 -> Pino 7 (Arduino UNO).

Isolamento de Domínios:

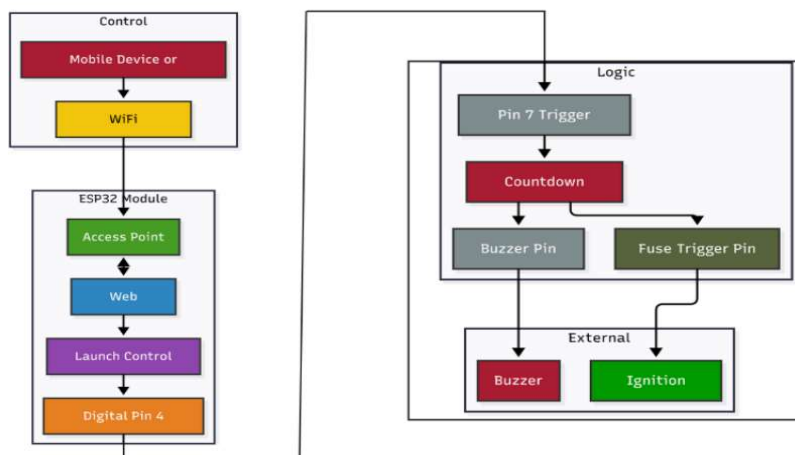
Desconexão física entre a pilha de protocolos Wi-Fi e o loop de atuação.

- > Mapeamento de hardware do Arduino UNO.
- > Percebam o uso da diretiva #define.

```
18  #define PIN_TRIGGER_INPUT  7  //  
19  #define PIN_PWM_FUSE      9  // F  
20  #define PIN_BUZZER        13  //
```

```
42  pinMode(PIN_TRIGGER_INPUT, INPUT_PULLUP);  
43  
44  // Saídas de atuação física  
45  pinMode(PIN_PWM_FUSE, OUTPUT);  
46  pinMode(PIN_BUZZER, OUTPUT);  
47
```

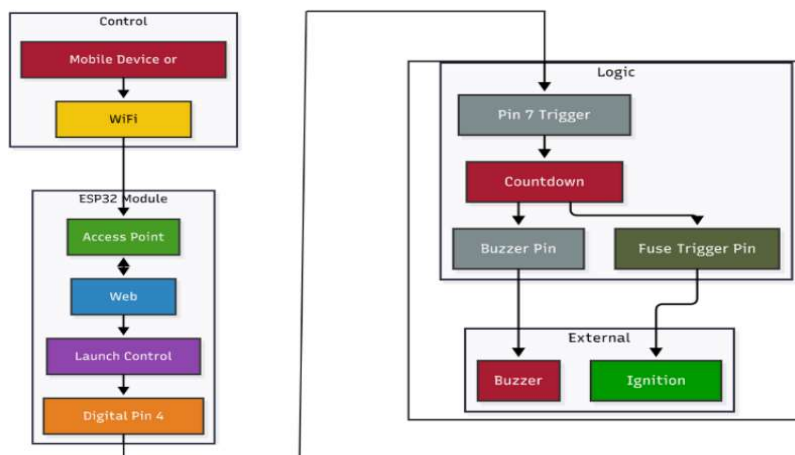
Discussão (Apresentando a Solução):



Tolerância a Falhas: Quedas ou travamentos na conexão sem fio não interferem na contagem ou no disparo.

Autonomia na Atuação: O atuador no Pino 9 opera de forma 100% determinística sob o controle da CPU do Uno.

Discussão (Apresentando a Solução):



Interlocks de Hardware e Software: Dupla verificação e trava pós disparo.

Baixa Ocupação do Barramento: Apenas 13,6% dos pinos GPIO totais do sistema são utilizados no projeto.

- > Dupla verificação como um filtro natural de debounce, transientes elétricos.
- > Trava pós trigger

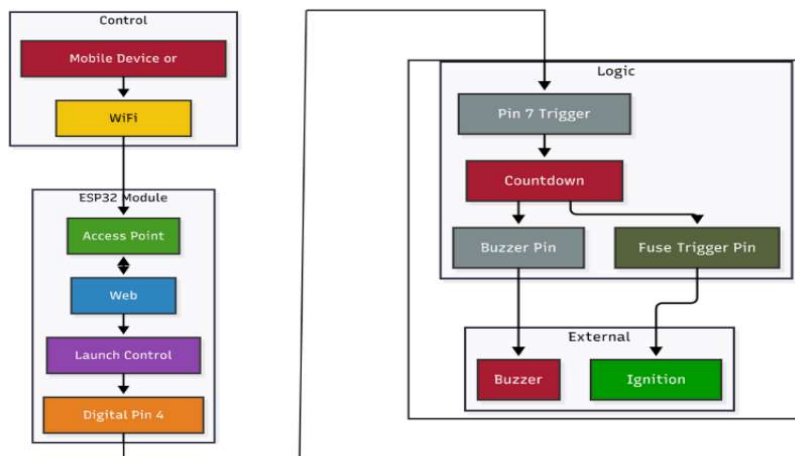
```
62     if (digitalRead(PIN_TRIGGER_INPUT) == LOW) {  
63  
64         // Pequeno debounce em nível de hardware para ignorar ruídos elétricos na  
65         delay(10);  
66  
67         if (digitalRead(PIN_TRIGGER_INPUT) == LOW) {  
68             Serial.println("\n>>> SINAL DE DISPARO DETECTADO NO PINO 7 <<<");  
69         }
```

```
138     while (digitalRead(PIN_TRIGGER_INPUT) == LOW) {  
139         delay(50);  
140     }
```

Desafio

Como você implementaria uma rotina de autodiagnóstico real no firmware? Considere toda a solução: tipo de sensores usados, comunicação com o MCU, implementação no firmware.

Discussão (Apresentando a Solução):



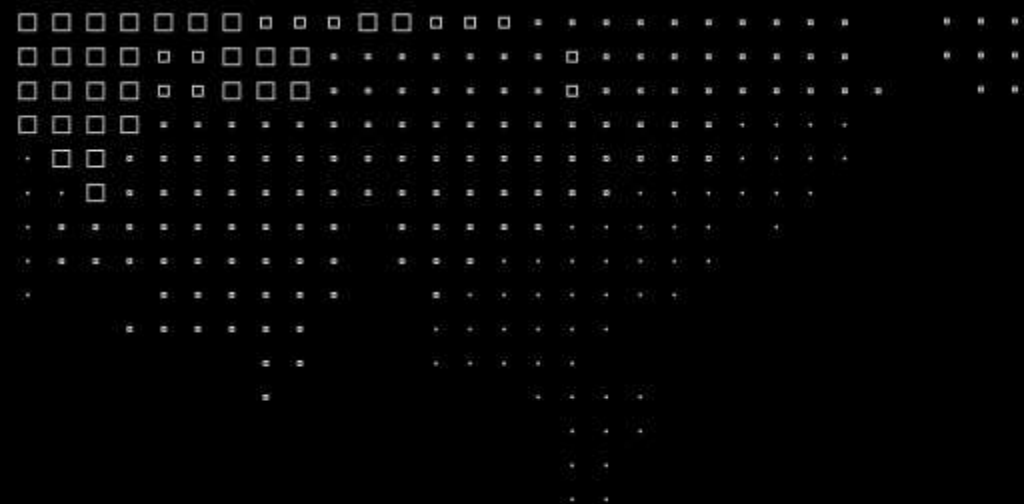
Simulação: Validação no ambiente Proteus 8 Professional com análise transiente e coupling analógico/digital.

Resultado: Zero acionamentos falsos e desvio temporal insignificante durante os testes.



Q&
A

DÚVIDAS?
SUGESTÕES?
COMENTÁRIOS



graduação⁺

Copyright © 2026 Prof. Hill

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).

